Tasks

Question 1

Implement a queue data structure using templates and perform basic operations on it.

Requirements:

1. Create a class called **Node using templates** to represent a node for a queue. Each node should have an integer data field and a pointer to the next node.

2. Create a class called **Queue using templates** to represent the queue data structure.

3. Queue will have a **front** pointer that will point towards the node in the front of the queue and a **rear** pointer that will point towards the end of the queue.

4. Implement the following methods in the **Queue** class:

    - **enqueue(item)**: Add an item to the back of the queue.
    - **dequeue()**: Remove and return the item from the front of the queue.
    - **peek()**: Return the item at the front of the queue without removing it.
    - **is_empty()**: Return **True** if the queue is empty, **False** otherwise.
    - **size()**: Return the number of elements in the queue.
    - **clear()**: Clear all elements from a node-based queue, effectively making it an empty queue.
    - **isPalindrome(char* str):** to check palindrome using recursion

Question 2

Multi-Level Feedback Queue Scheduler (Node-Based Queue)

Objective:

Design and implement a multi-level feedback queue scheduler in C++ using node-based queues. This scheduler will manage multiple queues with different priorities and use the First-Come, First-

Served (FCFS) scheduling algorithm within each queue. The task will involve creating classes and functions to simulate the scheduling process.

Task Description:

Process Class:

- Create a Process class to represent individual processes. Each process should have the following attributes:
    - Process ID
    - Priority (0 = high, 1 = medium, 2 = low)
    - Burst Time (the time it takes to execute the process)
    - Next pointer (for linking processes in the queue)
    - State (4 = terminated, 2=running)

CustomQueue Class:

- Implement a CustomQueue class for node-based queues. Each queue node should contain a Process object and a next pointer to the next node.
- Include the following member functions in the CustomQueue class:
    - isEmpty(): Check if the queue is empty.
    - enqueue(Process* process): Add a process to the queue.
    - dequeue(): Remove and return the process at the front of the queue.

MultiLevelFeedbackQueueScheduler Class:

- Create a MultiLevelFeedbackQueueScheduler class to manage the scheduling process.
- Include the following member functions in the MultiLevelFeedbackQueueScheduler class:
    - addProcess(Process process): Add a process to the appropriate priority queue based on its priority.
    - executeProcesses(): Implement the scheduling logic using FCFS within each priority queue and promote processes after a certain number of executions. Continues to execute processes within each priority level until the respective queue is empty before moving on to the next lower-priority queue.

Processes are promoted or demoted based on their priority level:

- High-priority processes are demoted to the low-priority queue.
- Medium-priority processes are promoted to the high-priority queue.
- Low-priority processes are promoted to the medium-priority queue.

```
Executing Process ID: 1 | Priority: 0 | Burst Time: 5 | State: Running
Process ID: 1 | Burst Time after execution: 3
Process ID: 1 | New Priority: 2 | Re-enqueued for next round.
Executing Process ID: 4 | Priority: 0 | Burst Time: 4 | State: Running
Process ID: 4 | Burst Time after execution: 2
Process ID: 4 | New Priority: 2 | Re-enqueued for next round.
Executing Process ID: 2 | Priority: 1 | Burst Time: 3 | State: Running
Process ID: 2 | Burst Time after execution: 1
Process ID: 2 | New Priority: 0 | Re-enqueued for next round.
Executing Process ID: 5 | Priority: 1 | Burst Time: 1 | State: Running
Process ID: 5 has finished execution and is terminated.
Executing Process ID: 3 | Priority: 2 | Burst Time: 2 | State: Running
Process ID: 3 has finished execution and is terminated.
Executing Process ID: 6 | Priority: 2 | Burst Time: 6 | State: Running
Process ID: 6 | Burst Time after execution: 4
Process ID: 6 | New Priority: 1 | Re-enqueued for next round.
Executing Process ID: 1 | Priority: 2 | Burst Time: 3 | State: Running
Process ID: 1 | Burst Time after execution: 1
Process ID: 1 | New Priority: 1 | Re-enqueued for next round.
Executing Process ID: 4 | Priority: 2 | Burst Time: 2 | State: Running
Process ID: 4 has finished execution and is terminated.
Executing Process ID: 2 | Priority: 0 | Burst Time: 1 | State: Running
Process ID: 2 has finished execution and is terminated.
Executing Process ID: 6 | Priority: 1 | Burst Time: 4 | State: Running
Process ID: 6 | Burst Time after execution: 2
Process ID: 6 | New Priority: 0 | Re-enqueued for next round.
Executing Process ID: 1 | Priority: 1 | Burst Time: 1 | State: Running
Process ID: 1 has finished execution and is terminated.
Executing Process ID: 6 | Priority: 0 | Burst Time: 2 | State: Running
Process ID: 6 has finished execution and is terminated.

Terminated Processes:
Process ID: 5 | Final Burst Time: 0 | State: Terminated
Process ID: 3 | Final Burst Time: 0 | State: Terminated
Process ID: 4 | Final Burst Time: 0 | State: Terminated
Process ID: 2 | Final Burst Time: 0 | State: Terminated
Process ID: 1 | Final Burst Time: 0 | State: Terminated
Process ID: 6 | Final Burst Time: 0 | State: Terminated
```